

Animationen mit Manim

Lehrveranstaltungsskript

June 16, 2025

Einleitung

Manim ist ein leistungsstarkes Python-Framework zur Erstellung mathematischer Animationen. Dieses Skript behandelt die wichtigsten Animationsklassen und Methoden in Manim, erläutert ihre Funktionsweise, gibt Beispiele und beleuchtet didaktische Hinweise für den Einsatz im Unterricht oder in Erklärvideos.

1 Grundlegende Animationen

1.1 Create

Die Methode **Create** zeichnet das Objekt Schritt für Schritt auf den Bildschirm. Sie eignet sich besonders für geometrische Objekte oder zur Hervorhebung neuer Inhalte.

Beispiel:

```
class CreateExample(Scene):
    def construct(self):
        square = Square(fill_color=BLUE, fill_opacity=0.5)
        self.play(Create(square))
        self.wait()
```

Die **Create**-Animation erzeugt das Objekt visuell, als würde es gezeichnet werden. Verwendbar mit allen Objekten, die von **Mobject** erben.

1.2 Write

`Write` zeigt ein Textobjekt so an, als würde es geschrieben – ideal für `Text`, `Tex` oder `MathTex`.

Beispiel:

```
class WriteExample(Scene):
    def construct(self):
        text = Text("Hallo Welt")
        self.play(Write(text))
        self.wait()
```

Die Zeichen erscheinen dabei nacheinander – ein Effekt, der besonders für Erklärtex te geeignet ist.

1.3 AddTextLetterByLetter und AddTextWordByWord

Diese Methoden stammen aus dem Zusatzpaket `manim_additions` und ermöglichen feinere Kontrolle über den Textaufbau. Alternativ kann man eigene Schleifen nutzen:

```
class LetterByLetter(Scene):
    def construct(self):
        text = Text("Hallo")
        for i in range(len(text)):
            self.play(FadeIn(text[i]))
        self.wait()
```

Dieser Stil wirkt modern und eignet sich gut für kurze, auffällige Videoausschnitte, z. B. für soziale Medien.

1.4 DrawBorderThenFill

`DrawBorderThenFill` zeichnet zunächst den Rand eines Objekts und füllt danach die Fläche. Dadurch entsteht eine elegante Zeichenanimation.

Beispiel:

```
class DrawFill(Scene):
    def construct(self):
```

```

circle = Circle()
self.play(DrawBorderThenFill(circle))
self.wait()

```

Besonders geeignet für Hervorhebungen oder Einführungen neuer geometrischer Objekte.

2 Weitere Animationen

2.1 ShowIncreasingSubsets

Diese Animation zeigt die Teilmengen eines Objekts nach und nach an – z.B. für Punktmengen oder Aufzählungen.

```

class IncreasingSubsetsExample(Scene):
    def construct(self):
        dots = VGroup(*[Dot(point=[x, 0, 0]) for x in range(-3, 4)])
        self.play>ShowIncreasingSubsets(dots)
        self.wait()

```

Ideal zur Visualisierung von Reihenfolgen oder schrittweisem Aufbau.

2.2 ShowPartial

ShowPartial ist eine niedrigere Ebene der Kontrolle und erlaubt es, nur einen Teil eines Objekts anzuzeigen. Dies erfordert Zugriff auf Unterobjekte oder Pfade.

2.3 ShowSubmobjectsOneByOne

Hierbei werden die Unterobjekte eines Mobjects (z. B. Buchstaben eines Textes) einzeln eingeblendet.

```

class SubmobjectsExample(Scene):
    def construct(self):
        word = Text("Hallo")
        self.play>ShowSubmobjectsOneByOne(word)
        self.wait()

```

Eine einfache Möglichkeit für Effekte wie Buchstabieren oder Listensymbole.

2.4 SpiralIn

`SpiralIn` lässt Objekte aus einer Spirale erscheinen. Diese Animation erzeugt einen dynamischen, spannungsvollen Effekt.

```
class SpiralInExample(Scene):
    def construct(self):
        square = Square()
        self.play(SpiralIn(square))
        self.wait()
```

2.5 TypeWithCursor

Animiert einen Texteffekt mit blinkendem Cursor, als würde live getippt.

```
class Typing(Scene):
    def construct(self):
        text = Text("Eingabe...")
        self.play(TypeWithCursor(text))
        self.wait()
```

Sehr effektiv für modern wirkende Screencasts oder zur Darstellung von Benutzereingaben.

2.6 Uncreate, Unwrite

Diese Methoden sind die Umkehrung von `Create` bzw. `Write`.

```
class RemoveExample(Scene):
    def construct(self):
        obj = Circle()
        self.play(Create(obj))
        self.wait()
        self.play(Uncreate(obj))
        self.wait()
```

Nützlich für Übergänge oder um Inhalte zu entfernen.

2.7 FadeIn, FadeOut

Blendet Objekte sanft ein oder aus. Standardanimation für viele Übergänge.

```
class FadeExample(Scene):
    def construct(self):
        square = Square()
        self.play(FadeIn(square))
        self.wait()
        self.play(FadeOut(square))
        self.wait()
```

2.8 Grow...

Animationen wie `GrowFromCenter` oder `GrowFromEdge` lassen Objekte aus einem bestimmten Punkt heraus erscheinen. Sehr nützlich zur Hervorhebung.

```
class GrowExample(Scene):
    def construct(self):
        circ = Circle()
        self.play(GrowFromCenter(circ))
        self.wait()
```

2.9 ApplyWave

Verleiht dem Objekt eine wellenartige Verzerrung. Gut für dramatische oder humorvolle Effekte.

```
class WaveExample(Scene):
    def construct(self):
        text = Text("Wellen")
        self.play(ApplyWave(text))
        self.wait()
```

2.10 Blink

Ein subtiler Effekt, bei dem das Objekt kurz aus- und eingeblendet wird, wie ein Blinzeln.

```
class BlinkExample(Scene):
    def construct(self):
        dot = Dot()
        self.add(dot)
        self.play(Blink(dot))
        self.wait()
```

2.11 Circumscribe

Circumscribe hebt ein Objekt hervor, indem es umrundet wird – ideal, um die Aufmerksamkeit gezielt zu lenken.

```
class CircumscribeExample(Scene):
    def construct(self):
        square = Square()
        self.add(square)
        self.play(Circumscribe(square))
        self.wait()
```

2.12 Flash

Flash erzeugt eine blitzartige Hervorhebung von Koordinaten oder Punkten – besonders hilfreich, um Punkte zu markieren.

```
class FlashExample(Scene):
    def construct(self):
        self.play(Flash(point=ORIGIN))
        self.wait()
```

2.13 FocusOn

FocusOn simuliert einen Lichteffekt, der ein Objekt in den Vordergrund hebt.

```
class FocusOnExample(Scene):
    def construct(self):
        square = Square()
        self.add(square)
        self.play(FocusOn(square))
        self.wait()
```

2.14 Indicate

Indicate animiert ein Objekt durch Pulsieren oder Skalierung – gut zur Betonung oder Wiederholung.

```
class IndicateExample(Scene):
    def construct(self):
        text = Text("Wichtig!")
        self.add(text)
        self.play(Indicate(text))
        self.wait()
```

2.15 ShowPassingFlash

ShowPassingFlash zeigt eine Animation, die ein Objekt entlang einer Linie oder Kurve passiert – etwa zur Darstellung von Bewegung.

```
class PassingFlashExample(Scene):
    def construct(self):
        line = Line(LEFT, RIGHT)
        self.add(line)
        self.play>ShowPassingFlash(line.copy().set_color(RED)))
        self.wait()
```

2.16 Wiggle

Wiggle lässt ein Objekt zittern oder vibrieren – nützlich für Unsicherheit oder Unruhe.

```
class WiggleExample(Scene):
    def construct(self):
        square = Square()
        self.add(square)
        self.play(Wiggle(square))
        self.wait()
```

2.17 ComplexHomotopy, Homotopy

Diese Methoden erlauben kontinuierliche Transformationen über eine benutzerdefinierte Funktion. Sie eignen sich z. B. für Verzerrungseffekte oder topologische Demonstrationen.

```
class HomotopyExample(Scene):
    def construct(self):
        square = Square()
        def wave(x, y, z, t):
            return x, y + 0.5 * np.sin(2 * np.pi * t + x), z
        self.play(Homotopy(wave, square))
        self.wait()
```

2.18 MoveAlongPath

Lässt ein Objekt einer Kurve folgen.

```
class PathExample(Scene):
    def construct(self):
        path = Circle()
        dot = Dot()
        self.play(MoveAlongPath(dot, path))
        self.wait()
```

2.19 PhaseFlow

PhaseFlow animiert ein Objekt entlang eines Vektorfeldes – ideal zur Darstellung dynamischer Systeme.


```

class PhaseFlowExample(Scene):
    def construct(self):
        func = lambda p: np.array([p[1], -p[0], 0])
        dot = Dot(RIGHT)
        self.add(dot)
        self.play(PhaseFlow(func, dot, run_time=3))
        self.wait()

```

2.20 rotate() vs. animate.rotate()

Manim unterscheidet zwischen sofortigen Methodenaufrufen und animierten Transformationen:

```

class RotateVsAnimate(Scene):
    def construct(self):
        square = Square()
        self.add(square)
        square.rotate(PI/4) # direkt, keine Animation
        self.wait()
        self.play(square.animate.rotate(PI/4)) # animiert
        self.wait()

```

Die Methode `rotate()` verändert das Objekt sofort. Erst `animate.rotate()` erzeugt eine Animation.

2.21 Transform-Animationen

`Transform`, `ReplacementTransform`, `FadeTransform` und `TransformMatchingTex` sind leistungsstarke Werkzeuge zur schrittweisen Umwandlung von Objekten.

Beispiel für Transform:

```

class TransformExample(Scene):
    def construct(self):
        square = Square()
        circle = Circle()
        self.add(square)
        self.play(Transform(square, circle))
        self.wait()

```

ReplacementTransform entfernt das Ursprungsobjekt automatisch:

```
self.play(ReplacementTransform(square, circle))
```

FadeTransform kombiniert Überblenden mit Veränderung:

```
self.play(FadeTransform(square, circle))
```

TransformMatchingTex eignet sich für mathematische Umformungen, bei denen einzelne Terme erhalten bleiben:

```
class TexTransform(Scene):
    def construct(self):
        expr1 = MathTex("a^2 + b^2")
        expr2 = MathTex("c^2")
        self.add(expr1)
        self.play(TransformMatchingTex(expr1, expr2))
        self.wait()
```

Abschließende Hinweise

Dieses Skript deckt eine Vielzahl nützlicher Animationsmethoden in Manim ab. Für die vollständige Liste siehe die Manim-Dokumentation unter <https://docs.manim.community>.